

A Rapid Opcode Development Workflow for Real-Time Physical Modelling in Csound

Juliette Tудоce^{1,2}, Vasileios Chatziioannou², and Alex Hofmann² *

¹ Laboratoire d'Acoustique de l'Université du Mans, Institut d'Acoustique – Graduate School

² Dept. of Music Acoustics, mdw – University of Music and Performing Arts Vienna
juliettetudoce@me.com, chatziioannou@mdw.ac.at, hofmann-alex@mdw.ac.at

Abstract. Implementing a custom physical modelling opcode in Csound requires fluency in C programming, a skill that remains a significant barrier for many musicians and researchers in music acoustics, despite the strong potential of physics-based sound synthesis. This paper presents a development workflow for creating a Csound opcode, illustrated through the case study of the Đàn Bầu, a Vietnamese monochord instrument plucked by the right hand and pitch-bended by the left hand through a stem that modulates the tension of the string. Starting from a generic string model drawn from the acoustics literature, we first prototyped it in Python, allowing for straightforward implementation and assessment of the energy balance. We then translated the Python code into C with the help of Large Language Models (LLMs), which produced a functionally correct translation in a single pass. Finally, the resulting C code was manually adapted into an opcode plugin, enabling its integration into Csound. This case study suggests that the complete process from mathematical model to performable instrument through Csound can be achieved without deep C programming expertise. The workflow presented here offers an illustrative example that musicians and researchers in music acoustics may draw on when developing other physical modelling opcodes for real-time implementation in Csound.

Keywords: csound, opcode, workflow, physical modelling, Vietnamese instruments

1 Introduction

Physical modelling offers an approach to digital instrument design in which sound generation is grounded directly in the underlying physical principles that describe how an instrument behaves. Csound supports this approach through its opcode architecture, allowing custom algorithms to be integrated directly into the language. However, the development of a custom Csound opcode generally requires fluency in C programming, familiarity with the Csound Plugin API [1], and a knowledge of numerical methods of solving partial differential equations (PDEs). This combination represents a significant barrier for musicians, composers, and researchers who may have strong intuition about the physical behaviour of an instrument but limited experience with programming languages. Our work contributes to the existing set of physical modelling opcodes [2,3], specifically targeting the Đàn Bầu.

This paper presents a development workflow, illustrated through a single case study, that aims to lower this barrier by pairing open-source reference material with the code-translation capabilities of Large Language Models (LLMs). The workflow proceeds in five stages: mathematical model, Python prototype, energy-based validation, LLM-assisted translation to C, adaptation to Csound Plugin framework. The LLM acts as a translation layer between the validated high-level code and the low-level implementation needed by the Csound architecture.

The workflow is illustrated through the example of the Đàn Bầu, a Vietnamese monochord instrument with a pitch-bend technique. The resulting opcode `danbau` implements a finite difference simulation of a stiff damped string with spatially localised hand damping and external tension modulation. The paper is organised as follows: Section 2 describes the development workflow in detail. Section 3 presents the case study, covering the instrument, its physical model, the opcode development, and a performance example. Finally, Section 4 discusses limitations, design choices, and directions for future work.

* This research was funded in part by the Austrian Science Fund (FWF) [10.55776/AR743].

2 Implementation Workflow

The development of the **danbau** opcode from the mathematical formulation to a performable instrument was done in 5 stages, each one building directly on the output of the previous one. The use of LLMs at specific steps significantly reduced the time and low-level coding expertise required.

- The starting point was the stiff damped string model as presented by Bilbao [4] in continuous and discrete form. This model was first implemented manually in Python for its simplicity and availability of scientific libraries. Python allowed us to freely experiment with the parameters and visualise the string displacement at each step through animations.
- Before proceeding to any translation, the Python implementation was validated through an energy balance analysis. The total mechanical energy (kinetic plus potential) was monitored across the simulation. In a lossless system this quantity should remain constant, while in the presence of damping it should decay monotonically. We also made sure that the stability condition was not violated for the chosen parameters. This step was also carried out manually.
- Once the Python prototype was validated, the code was translated into C using an LLM. Rather than rewriting the implementation from scratch, the Python source was provided to the model with the following prompt:

"Translate the following Python implementation of a finite difference simulation into a C code. Do not change the underlying algorithm or simplify the physics."

The model produced a functionally equivalent C implementation in a single pass, correctly mapping NumPy array operations to explicit loops and preserving the finite difference structure. The same energy balance analysis as above was carried out on the obtained C implementation.

- The standalone C code was then adapted into a Csound opcode using the Csound API documentation [5,1]. This step required wrapping the simulation inside the **OPDS** structure, separating initialisation from performance-time logic into **danbau_init** and **danbau_perf** functions respectively, correctly declaring input and output port types following Csound's type string convention. Minor adjustments and debugging, as well as some compilation commands, were once again delegated to an LLM, which was prompted with the validated C code as follows:

"Here is my Csound opcode implementation, adapted from a standalone C simulation. Review it for errors, fix any issues preventing it from compiling or running correctly, and provide the gcc command needed to compile it as a dynamic library against the CsoundLib64 framework."

- The resulting plugin was compiled and loaded into Csound at runtime using the **-opcode-lib** flag. A Csound Instrument was then written to drive the simulation.

3 The Đàn Bàu: a Case Study

3.1 Đàn Bàu

The Đàn Bàu is a Vietnamese monochord instrument, introduced to the court at the end of the 19th century [6]. It consists of a steel string strung obliquely across a rectangular wooden soundboard. At one end, it is tied to a perpendicular, flexible, bamboo stem that can be bent by the performer's left hand to alter the string's tension and produce pitch-bending. The right hand plucks the string with a long plectrum and damps the string simultaneously with the back side of the little finger to produce harmonics. The *electric Đàn Bàu* uses a magnetic pickup placed near the bridge to amplify the sound. This opcode specifically supports the relevant string parameters of this instrument.

3.2 Physical model

From a physical modelling perspective, the Đàn Bàu can be seen as a damped stiff string which can be modelled by the following PDE:

$$\rho A u_{tt} = T u_{xx} - E I u_{xxx} - \sigma_0 u_t - \sigma_1 u_{xt} + f, \quad (1)$$

where $u(x, t)$ denotes the transverse displacement of the string at position x and time t . The parameters ρ , A and T represent respectively the mass density, the cross-sectional area, and the tension of the string. $E I$ is the bending stiffness, characterised by the product of the Young modulus E and the second moment of area I . σ_0 and σ_1 denote the frequency-independent and frequency-dependent damping coefficients, respectively. The external force f represents the excitation applied to the string, modelling the performer's pluck. The numerical model follows the finite difference time domain framework (FDTD) [4] to update the string displacement at each discrete time step.

Pluck excitation Following [7], the plucking force is modelled as

$F(t) = F_p \sin^2 \left[\frac{\pi}{d} \frac{1}{2} \left(t + \frac{d \sinh(\beta t/d)}{\sinh(\beta)} \right) \right]$, $0 \leq t \leq d$, where F_p is the force amplitude, d is the pluck duration, and $\beta > 0$ controls the stiffness of the pluck. Since the pluck position does not necessarily coincide with a grid point, the force is distributed onto the nearest neighbouring grid points, using linear interpolation weights.

Hand damping When playing the Đàn B  u, the performer uses the right hand to lightly touch the string near the pluck point, shaping the harmonic content. While global frequency-dependent and independent damping coefficients are applied uniformly across the string, we incorporate the hand damping into the physical model by introducing spatially localised damping terms concentrated around the hand position. Concretely, this is implemented by locally increasing the damping coefficients at the grid points nearest to the hand position. The localised damping is distributed across up to 4 neighbouring grid points using cubic interpolation weights. The localised damping terms are activated simultaneously with the pluck and remain effective for a controlled duration, which may exceed the pluck duration.

Tension modulation String tension directly governs the wave propagation speed $c = \sqrt{T/\mu}$, where $\mu = \rho A$ is the linear mass density. Variations in tension alter the propagation speed and, consequently, the modal frequencies of the string. Time-dependent tension modulation therefore produces continuous pitch variations, allowing the characteristic Đàn B  u stem feature to be reproduced. In the current implementation, tension is updated at each control-rate step and the finite difference coefficients are recomputed accordingly, while the grid itself is kept fixed. The grid spacing and time step are dimensioned once, at initialisation, so that the stability condition is satisfied across the entire tension range supported by the opcode, typical of Đàn B  u playing. We note, however, that more rigorous treatments of time-varying parameters exist [8,9]. In fact, more complex models for plucking, damping, string nonlinearity, and tension modulation are available (see, e.g., [8,9,10]). These are omitted here for simplicity, since this work focuses on the opcode development workflow.

3.3 Opcode Signature and Output

The Csound opcode is named `danbau`, allowing the user to control relevant performance parameters, while keeping other physical parameters fixed.³ These performance parameters were chosen because they correspond to the physical quantities a performer directly manipulates during play (string tension via the stem, and hand damping duration/magnitude). The opcode signature is

```
aout danbau ktension, iposition, kmag_damp, kdur_damp
[, iforce_pluck, idur_pluck, ihand, ibeta]
```

where `ktension` controls the string tension in Newtons (between 50 and 900 N), `iposition` sets the damping position as a fraction of the string length (0 at the nut, 1 at the bridge), `kmag_damp` and `kdur_damp` define the magnitude (0 without damping, 1 strong damping) and the duration in seconds of the hand damping, `iforce_pluck` and `idur_pluck` define the force (between 0 and 1, default: 0.8 N) and the duration in seconds (default: 0.002 s) of the pluck excitation, `ihand` sets the distance of the performer’s damping hand from the plucking point, in meters (default: 0.06 m), and `ibeta` sets the stiffness parameter β , which shapes the temporal asymmetry of the pluck-force envelope defined in Section 3.2 (default: 3.0). The output `aout` is a single audio-rate signal corresponding to the normalised string velocity evaluated at the pickup position. As a further validation step, the output waveforms generated by the Csound opcode were compared against those produced by the original Python implementation for a few representative test cases, showing consistent agreement between the two. These waveform comparisons are not included here for space reasons.

Figure 1 presents the magnitude spectra corresponding to the notes defined in this Csound score: `<CsScore>`

```
; inst  start  dur  tension  position  mag_damp  dur_damp  force_pluck  dur_pluck
i 1      0     10   381      [1/2]     0         0         0.8         0.002
i 1     10     10   381      [1/2]     1        0.02     0.8         0.002
i 1     20     10   381      [1/2]     1         1         0.8         0.002
i 1     30     10   381      [2/3]     1         1         0.8         0.002
</CsScore>
```

³ <https://github.com/jtudoce/Dan-bau-Opcode-Csound>

Panel (a) corresponds to a conventional pluck without hand damping, located at the middle of the string, and exhibits a rich harmonic structure. Panels (b) and (c) correspond to the same plucking position, with hand damping durations of 0.02 s and 1 s respectively. The fundamental frequency and even harmonics are attenuated under brief damping and annihilated under long damping. Panel (d) can be compared with panel (c). It corresponds to the same 1 s damping duration but with the damping position located at $x = 2L/3$, yielding a natural harmonic corresponding to a perfect fifth. In this case, only the harmonics whose order is a multiple of three persist, whereas all other components are suppressed. These examples show that both the position and the duration of the hand damping influence the harmonic content of the resulting note, creating a Đàn Bầu related sound timbre.

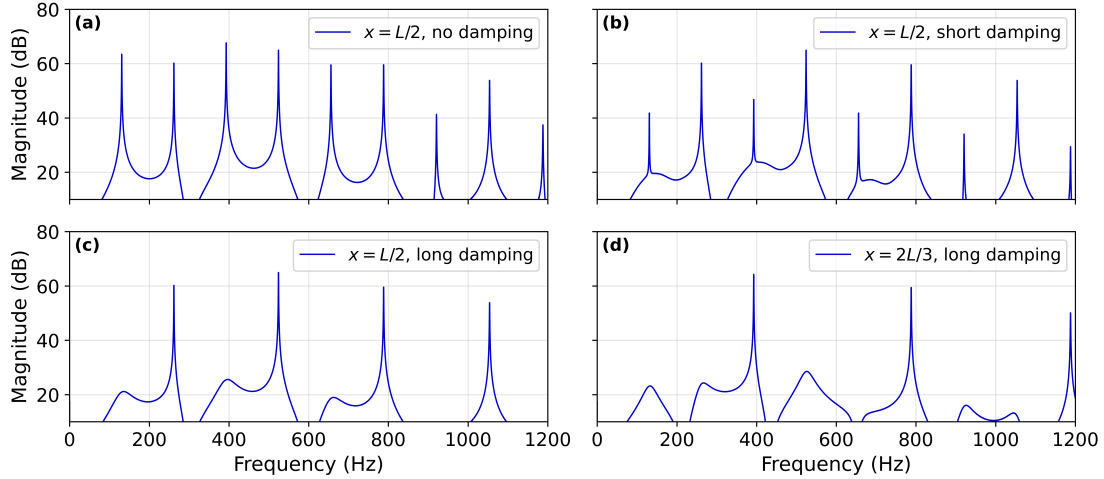


Fig. 1. Magnitude spectra obtained for four damping configurations. (a) Ordinary pluck in the middle of the string without the hand damping. (b) Harmonic produced with a very brief hand damping at position $L/2$. (c) Harmonic produced with a prolonged hand damping at position $L/2$. (d) Harmonic produced with a prolonged hand damping at position $2L/3$. The tension is fixed at 381 N, the plucking force and duration are respectively 0.8 N and 0.002 s, the hand position is fixed at 0.06 m, and $\beta = 3.0$.

4 Discussion

The workflow presented in this paper demonstrates that the use of LLMs can help in the development of physical modelling opcodes for Csound, despite limited prior experience with C programming. In the present case, they assisted the authors in translating a Python prototype into C and integrating the resulting code into the Csound plugin framework. However, using LLMs does not remove the need for a basic understanding of programming concepts, numerical methods and musical acoustics. Although the generated code was often correct, manual intervention remained necessary to inspect the implementation, identify errors, debug compilation issues, and adapt the code to the Csound API's requirements. In practice, understanding the structure and behaviour of the generated code remains essential, especially when modifications or extensions are required.

It should be noted that the workflow's efficiency claims rest on a single case rather than a controlled evaluation: we report no wall-clock time, no comparison against a manual C translation, and no data on prompt iterations, model errors, or how they were identified and corrected. The two prompts quoted in Section 2 document what was asked of the LLM, but do not establish that the workflow is reproducible or generalisable to other physical models or developers. We therefore present this workflow as a proof of concept rather than a validated method, and see a systematic evaluation across multiple case studies and developers with varying C proficiency as future work.

More generally, the assistance by LLMs raises questions regarding scientific production. These systems enable the rapid production of an increasing number of ideas, prototypes, and publications. Nevertheless, the outputs still require careful human review and validation. The quantity of generated material grows way faster than the ability of researchers to evaluate it critically, making it increasingly difficult to keep pace with the expanding literature.

Future work will focus on extending the opcode for live performance with a MIDI controller. In particular, a gestural control interface inspired by Grund et al. [11] may facilitate expressive interpretation during performances. More broadly, the workflow described here could be applied to other acoustic systems, contributing to a larger ecosystem of custom physical modelling opcodes for Csound.

References

1. ffitc, J.: Extending Csound, pp.599–612. In: Boulanger, R. (ed.) *The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing and Programming*. MIT Press, Cambridge, MA (2000)
2. Lazzarini, V., Yi, S., ffitc, J., Heintz, J., Brandtsegg, Ø., and McCurdy, I.: Physical Models, pp.385–405. In: *Csound: A Sound and Music Computing System*. Springer International Publishing, Cham (2016)
3. Hofmann, A., Schmutzhard, S., Pàmies-Vilà, M., Erdogan, G., Chatziioannou, V.: An opcode implementation of a finite difference viscothermal time-domain model of a tube resonator for wind instrument simulations. In: Proc. Fifth International Csound Conference (2019)
4. Bilbao, S.: *Numerical Sound Synthesis: Finite Difference Schemes and Simulation in Musical Acoustics*. John Wiley & Sons (2009)
5. Csound API documentation, <https://csound.com/docs/api/index.html>
6. Trần Q. H.: Đàn Bầu, pp. 540–541, In: Sadie, S. (ed.) *The New Grove Dictionary of Musical Instruments, Vol. 1 (A to F)*, Macmillan Publishers, London (1984)
7. van Walstijn, M., Bridges, J., Mehes, S.: A Real-Time Synthesis Oriented Tanpura Model. In: Proc. Int. Conf. Digital Audio Effects. Brno (2016)
8. Willemsen, S., Bilbao, S., Ducceschi, M., Serafin, S.: The dynamic grid: Time-varying parameters for musical instrument simulations based on finite-difference time-domain schemes. *Journal of the Audio Engineering Society*, 70(9), pp.650–660 (2022)
9. van Walstijn, M., Chatziioannou, V., Bhanuprakash, A.: Tunable collisions: Hammer-string simulation with time-variant parameters. In: Proc. Int. Conf. Digital Audio Effects. Copenhagen (2023)
10. Russo, R.: Non-iterative numerical simulation techniques for nonlinear string vibration in musical acoustics. PhD thesis, University of Bologna (2025)
11. Grund, T.-T., Lương, H. T., Hofmann, A.: Challenges and Prospects in Remote Cross-cultural Musical Interface Design. In: Proceedings of the International Conference on New Interfaces for Musical Expression, pp.447–451. Utrecht (2024)